

Core Payment Project

We've build this project as an senior Android developer assessment and example usage of **Android IPC, TCP networking, local data persistence, concurrency control, foreground services** and etc. The system consists of two independent Android apps communicating over **AIDL**.

AIDL is an acronym for **Android Interface Definition Language**. It's Androids' tool for talking between two separate process (mostly apps) since normal apps can't just directly call a function from another app, in our project `merchant app` and `payment core app` are two distinct process, so they communicate over AIDL.

How's the AIDL contract works?

All `.aidl` files should be in an shared module accessible by both apps, so both apps depend on it via:

```
implementation(project(":aidl_contract"))
```

This is why we put all of our `.aidl` files in a shared module called `aidl_contract`.



We have two model data classes, they're Kotlin classes marked as **Parcelable**.

`PaymentReqeust.kt`:

```
@Parcelize
data class PaymentRequest(
    val requestId: String,
    val amount: Long,
    val terminalId: String,
```

```

        val traceNumber: Long,
        val keepServiceAlive: Boolean = true
    ) : Parcelable

```

PaymentResult.kt:

```

@Parcelize
data class PaymentResult(
    val requestId: String,
    val status: String,
    val responseCode: String?,
    val rrn: String?,
    val message: String?,
    val durationMs: Long
) : Parcelable

```

Parcelable is Android's own lightweight serialization format built for speed across IPC. Kotlin/Java `Serialization` uses class reflection and is heavy allocation, bad for something that has to happen on every binder call. *remember both of them (Parcelable, Serialization) are ways to turn objects into bytes so that data can cross IPC.*

```

parcelable PaymentRequest;
parcelable PaymentResult;

```

These are just format declarations telling the AIDL compiler that "this file exists, and it's Parcelable" we need this files so the other `.aidl` files can reference to our Kotlin data classes. without theses files + Parcelable Kotlin data classes are useless to AIDL compiler.

IPaymentService.aidl:

```

interface IPaymentService {

    void startTransaction(in PaymentRequest request, IPaymentCallback callback);

    PaymentResult getTransactionStatus(String requestId);

}

```

This file is the main contract. `merchant` app would implement it and call this functions to get transaction data or embark one.

- in `PaymentRequest` request the `in` keyword is a directionally tag required by AIDL compiler for any non-primitive parameter.
- `getTransactionStatus` is a blocking synchronous call, the caller waits for `PaymentResult` to come back directly

`IPaymentCallback.aidl` :

```
oneway interface IPaymentCallback {

    void onTransactionStarted(String requestId);

    void onTransactionProgress(String requestId, String status);

    void onTransactionComplete(in PaymentResult result);

    void onTransactionFailed(in PaymentResult result);

}
```

The intension is to push events to `merchant` they are just callbacks.

- `oneway interface` a `oneway` method is **asynchronous/fire-and-forget**. The caller returns immediately on the caller's thread without waiting for any work to finish. **So obviously no value return is possible here** this is why all functions in `IPaymentCallback` return `void`.

Core Payment App

This app is the heart of the our living project, app binds services, saves local data, makes TCP connections, sends notification in foreground service and more...

here binder class is important

`binder/PaymentServiceBinder.kt` :

```
class PaymentServiceBinder @Inject constructor(
    private val startTransactionUseCase: StartTransactionUseCase,
    private val getTransactionUseCase: GetTransactionUseCase,
    private val notificationHelper: NotificationHelper
) : IPaymentService.Stub()
```

When you write a `.aidl` file Android generates Stub class for you, like:

`IPaymentService.aidl --> IPaymentService.Stub()`

This class is literally a subclass of `android.os.Binder` so when you extend it your class now is a subclass of binder too. This is the same object that handed back from `Service.onBind()`:

```
override fun onBind(p0: Intent?): IBinder = binder
```

This class is the Binder implementation that the OS uses to route incoming cross-process calls, every method `merchant` app calls remotely lands here! and executes in **Binder thread pool**.

We have injected use cases here to maintain clean architecture, Binder class is not responsible for notifications, TCP connection or any transaction data.

As mentioned before functions here like `startTransaction()` is called on Binder thread pool which Android manages it this thread is synchronous by default means caller expects the result immediately but our `StartTransactionUseCase` is a flow type with unhandled TCP connection delays this would exhaust Binder thread so instead we use:

```
private val binderJob = SupervisorJob()
private val binderScope = CoroutineScope(Dispatchers.IO + binderJob)
```

`startTransaction()` launches a coroutine and returns immediately, freeing the Binder thread, this coroutine scope needs a lifecycle to tied to so no leaks happens when service is gone when `Service.onDestroy()` fires coroutine gets cancelled.

```
override fun onDestroy() {
    binder.cancelJob()
    super.onDestroy()
}
```

`getTransactionStatus` uses `runblocking` instead of launching coroutine, this will block Binder thread (not main thread) to get result from synchronous oneway AIDL interface method call.

`AtomicBoolean` has been preferred over mechanisms like `ReentrantLock`, `synchronized`, or `Coroutine Mutex` since we get the single transaction at time in most simple and reliable way. `Mutex` also could be a good choice but we are not ordering transactions, we reject the coming transaction if another is in progress.

`PaymentCoreService.kt` :

```

@AndroidEntryPoint
class PaymentCoreService : Service() {

    @Inject lateinit var binder: PaymentServiceBinder
}

```

Service is an Android base framework for components that run in the background without visible UI. This class gets registered in app manifest and starts up by OS, because of that hilt also needs `@AndroidEntryPoint` annotation. Hilt also cannot inject in this class OS handles constructor so we use the power of Kotlin + Hilt to get `PaymentServiceBinder` as a field inject.

Inside the Services' `oncreat()` one of the important design decisions lays. Here each method is a thin wrapper around a real Android service Api and `PaymentServiceBinder` works with this interface and its anonymous object implementation.

```

override fun onCreate() {
    super.onCreate()

    val notificationLifecycle = object : TransactionNotificationServiceLifecycle {

        override fun enterForeground(notification: Notification) {
            startForeground(NotificationHelper.NOTIFICATION_ID, notification)
        }
        override fun updateForeground(notification: Notification) {
            notificationManager.notify(
                NotificationHelper.NOTIFICATION_ID, notification
            )
        }
        override fun exitForeground() {
            stopForeground(STOP_FOREGROUND_REMOVE)
        }
        override fun stop() {
            stopSelf()
        }
    }

    binder.attach(notificationLifecycle)
}

```

```
}
```

- `enterForeground()` -> `startForeground()` This is Androids' Api which promotes the running service into **foregroundService**, *foregroundService always shows a related notification to user and tells OS to not kill me!*
- `updateForeground()` -> `notify()` Updates the existing notification content that `startForeground()` made using the same notification id. *this will stop retriggering startForeground() each time*
- `exitForeground()` -> `stopForeground()` removes notification and turn the foreground to normal service
- `stop()` -> `stopSelf()` tells OS to remove the service entirely

why go through the interface instead of just calling `startForeground()` and other service methods directly inside `PaymentServiceBinder` ?

Because `startForeground()` and other service methods only exist in `Service()` but `PaymentServiceBinder` is not a service

```
binder.attach(notificationLifecycle)
```

This hands out anonymous implementation over to `PaymentServiceBinder` , filling its `lateinit` var

```
override fun onBind(p0: Intent?): IBinder = binder
```

This is the method that OS calls when another process binds to this service via `bindService()` . Whatever `IBinder` you return here is what client gets back as the object it can call AIDL methods.

core payment app provide TCP connection through a class called `TcpConnection.kt`

```
class TcpClient @Inject constructor() {  
  
    companion object {  
        private const val HOST = "192.168.1.6"  
        private const val PORT = 9999  
        private const val CONNECTION_TIMEOUT = 5_000  
        private const val READ_TIMEOUT = 10_000  
    }  
}
```

```

suspend fun openConnection(): TcpConnection = withContext(Dispatchers.IO) {
    val socket = Socket()
    try {
        socket.connect(InetSocketAddress(HOST, PORT), CONNECTION_TIMEOUT)
        socket.soTimeout = READ_TIMEOUT
        TcpConnection(socket)
    } catch (e: SocketTimeoutException) {
        runCatching { socket.close() }
        throw GatewayException.ConnectionTimeout(e)
    } catch (e: IOException) {
        runCatching { socket.close() }
        throw GatewayException.ConnectionFailed(e)
    }
}
}

```

after successful TCP connection `openConnection()` returns you an object of `TcpConnectionImpl` class so you can communicate with TCP server

```

class TcpConnectionImpl(private val socket: Socket) : TcpConnection {

    override suspend fun send(request: GatewayRequest) { ... }

    override suspend fun receive(requestId: String) { ... }

    override fun close() { ... }

}

```

Usage of these are in `StartTransactionUseCase` .

Merchant App

So simple, `PaymentCoreConnector` connects everything and let's this app to call AIDL methods it's mirror client-side of `PaymentCoreBinder` . This object should be single for whole app process since we don't want multiple connectors handling transactions at the same time which this responsibility has been dedicated to Hilt with `@singletone` annotation.

```
@Singleton
class PaymentCoreConnector @Inject constructor(
    @ApplicationContext private val context: Context
)
```

```
private var service: IPaymentService? = null
```

Service field is null at first means: nothing has connected yet we fill this in

`onServiceConnected()` once filled this holds the live AIDL proxy we can call methods on it

```
private var bindDeferred: CompletableDeferred<IPaymentService> =
    CompletableDeferred()
```

CompletableDeferred is a coroutine representing a future value that isn't ready yet, this solves a real timing problem: `bind()` is a asynchronous call it doesn't mean the service is connected yet (that happens later on a callback) so if some code calls `startTransaction()` via `service` field before the binding completes, it needs to suspend and wait until connection is ready rather than falling with `NullPointerException`, for service complete `CompletableDeferred` can await it.

```
private val connection = object : ServiceConnection {

    override fun onServiceConnected(name: ComponentName?, binder: IBinder?) {
        val paymentService = IPaymentService.Stub.asInterface(binder)
        service = paymentService
        if (!bindDeferred.isCompleted) bindDeferred.complete(paymentService)
    }

    override fun onServiceDisconnected(name: ComponentName?) {
        service = null
        bindDeferred = CompletableDeferred()
    }

}
```

Once you call `bindService()` OS calls the callbacks from `ServiceConnection` interface, `asInterface(binder)` is AIDL generated code that wraps the raw binder into a usable `IPaymentService` proxy.

Inside the `merchant` app application class every time in `onStart()` we bind our explicit intent

matching the `core payment` app manifest.

every time in `onStart()` :

```
fun bind() {
    val intent = Intent("info.alihabibi.payment_core.ACTION_BIND_PAYMENT").apply {
        setPackage("info.alihabibi.payment_core")
    }
    context.bindService(intent, connection, Context.BIND_AUTO_CREATE)
}
```

every time in `onStop()` :

```
fun unbind() {
    runCatching { context.unbindService(connection) }
    service = null
}
```

for proper binding we need some **queries** in `merchant` app manifest, this queries let both apps see their package name and communicate, **without queries the binding would always fail**

```
<queries>
  <package android:name="info.alihabibi.payment_core" />
</queries>
```

```
fun startTransaction(request: PaymentRequest): Flow<PaymentEvent> = callbackFlow {
    ... }
}
```

`callbackFlow` is a coroutine flow builder designed to bridge a old-styled callback APIs into flow. AIDL callbacks are old-styled callbacks but our components like view model wants to consume this events reactively as flow. `callbackFlow` will give you this which you can call `trySend()` from inside callback to emit values downstream. Inside `startTransaction()` we actually implemented the `IPaymentCallback.Stub()` and simply push transaction events by using `trySend()` .



Finally this class has fallen into repository to maintain repository pattern and view model uses this pattern to start/get transactions on button click events.





Ali Habibi, 8/24/2026